Bachelor's Thesis Computing Science



Radboud University Nijmegen

Thesis Title

Subtitle if you like

Author:
Floris Reuvers
s1096976

First supervisor/assessor:
Dr. Niels van der Weide

Second assessor:
Dr. Engelbert Hubbers

June 5, 2025

Abstract

Brief outline of research questions, results. (The preferred size of an abstract is one paragraph or one page of text.)

Contents

1 Introduction **2**

2 Lambda Calculus **3**
2.1 Introduction to the $\boldsymbol{\lambda}$-calculus . . . 3
2.2 Well-Typedness . . . 3
2.3 $\boldsymbol{\beta}$-Reduction . . . 5
2.3.1 Free and Bound Variables . . . 5
2.3.2 Substitution . . . 6
2.3.3 $\boldsymbol{\alpha}$-equivalence . . . 7
2.3.4 $\boldsymbol{\beta}$-Reduction . . . 8
2.4 Call-by-name $\boldsymbol{\lambda}$-calculus . . . 8
2.5 Call-by-value $\boldsymbol{\lambda}$-calculus . . . 10

3 Call-by-box Lambda Calculus **12**
3.1 Specification of $\boldsymbol{\lambda_b}$ . . . 12
3.1.1 Intuitionistic logic IS4 . . . 12
3.1.2 Terms and Types of $\boldsymbol{\lambda_b}$ . . . 12
3.2 Substitution and Free and Bound variables in λ_b . . . 14

4 Embeddings in $\boldsymbol{\lambda_b}$ **18**
4.1 Girard's Embedding . . . 18
4.2 Embedding of $\boldsymbol{\lambda_n}$ into $\boldsymbol{\lambda_b}$ . . . 18
4.3 Gödel's Embedding . . . 19

5 Formalisation in Agda **21**
5.1 Box Calculus . . . 21
5.2 Linear Logic . . . 21

6 Related Work **22**

7 Conclusions **23**

A Appendix **25**

Chapter 1

Introduction

The introduction of your bachelor thesis introduces the research area, the research hypothesis, and the scientific contributions of your work. A good narrative structure is the one suggested by Simon Peyton Jones [**?**]:

- describe the problem / research question
- motivate why this problem must be solved
- demonstrate that a (new) solution is needed
- explain the intuition behind your solution
- motivate why / how your solution solves the problem (this is technical)
- explain how it compares with related work

Close the introduction with a paragraph in which the content of the next chapters is briefly mentioned (one sentence per chapter).

Starting a new paragraph is done by inserting an empty line like this.

Chapter 2

Lambda Calculus

This section provides a detailed and formal description of the λ-calculus. We define a formal grammar of the λ-calculus and give examples of some terms in the λ-calculus. Then we explain β-reduction, after which we define call-by-name evaluation and call-by-value evaluation.

2.1 Introduction to the λ-calculus

The grammar of the lambda calculus is as follows:

Definition 2.1 (λ-calculus)**.** The terms of the λ-calculus are given by:

$$M, N, P, Q \quad ::= \quad x \mid \lambda x.M \mid MN$$

Terms are denoted by M, N, P or Q and can either be of the form x, $\lambda x.M$ or MN:

- x is a variable, which is a symbol that represents an input.
- $\lambda x.M$ is an abstraction. An abstraction is an anonymous function, where x is the parameter and M is the body of the function.
- MN is a function application, where M and N are terms.

The λ-term $\lambda x.x$ is an abstraction. This specific abstraction is called the identity function and has one input parameter, namely x, and returns the input x. The body of the function is also x in this case and the function is often abbreviated as **I**. The λ-term $(\lambda x.x)y$ is a function application. So the identity function is applied to the variable y.

2.2 Well-Typedness

This section discusses the concept of well-typedness in the context of lambda calculus. We first discuss types, typing judgements and typing rules. We then explore how to use these rules to show that a λ-term is well-typed.

Types of λ-calculus are defined in Definition 2.2, and are either X, which is the base type, or a function type, $A \to A'$ with A and A' being types in λ-calculus.

Definition 2.2. Types of λ-calculus

$$A \quad ::= \quad X \mid \ A \to A'$$

A typing judgement is of the form $\Gamma \vdash M : A$, and it means that we can derive that M has type A from the assumptions in context Γ. A context Γ is a list of variable-type pairs, where each variable is associated with a type. A context is written as $\Gamma = x_1 : A_1, \ldots, x_n : A_n$, where each x_i is a variable and each A_i is a type.

The typing rules are defined in Definition 2.3. The Id rule states that if x has type A in the context, we can conclude that x has type A. The $\to$-I rule states that if we have a term M of type A' in the context $\Gamma, x : A$, we can conclude that from context Γ it follows that $\lambda x.M$ has type $A \to A'$. The $\to$-E rule states that if we have a term M of type $A \to A'$ and a term N of type A, we can conclude that the application MN has type A'.

Definition 2.3. Typing rules of λ-calculus

$$\frac{}{\Gamma, x : A \vdash x : A}\ \text{Id}$$

$$\frac{\Gamma, x : A \vdash M : A'}{\Gamma \vdash \lambda x.M : A \to A'}\ \to\text{-I}$$

$$\frac{\Gamma \vdash M : A \to A' \qquad \Gamma \vdash N : A}{\Gamma \vdash MN : A'}\ \to\text{-E}$$

A λ-term is considered to be *well-typed* if we can derive a typing judgement with the typing rules. The λ-terms that are well-typed can be assigned a type. In Example 2.4, we show that λ-term $(\mathbf{I}(\lambda x.\mathbf{I}x))$ is well-typed by giving the full typing derivation.

Example 2.4. Well-typedness of $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{I}\mathbf{I})$

$$\dfrac{\dfrac{\dfrac{}{y : A \to A \vdash y : (A \to A)}\ \text{Id}}{\vdash \lambda y.y : (A \to A) \to (A \to A)}\ \to\text{-I} \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{x : A, z : A \vdash z : A}\ \text{Id}}{x : A \vdash \lambda z.z : A \to A}\ \to\text{-I} \qquad \dfrac{}{x : A \vdash x : A}\ \text{Id}}{x : A \vdash \mathbf{I}x : A \to A}\ \to\text{-E}}{\vdash \lambda x.\mathbf{I}x : A \to A}\ \to\text{-I}}{\vdash \mathbf{I}(\lambda x.\mathbf{I}x) : A \to A}\ \to\text{-E}$$

In contrast to the λ-term $(\mathbf{I}(\lambda x.\mathbf{I}x))$, the lambda term MM cannot be assigned a type. Since MM is an application, we can only apply the $\rightarrow$-E rule, which would give a contradiction. The λ-term M cannot be both a function and the argument to the same function.

2.3 β-Reduction

To include a notion of computation to the λ-calculus, we discuss β-reduction. This reduction relation is the only way to simplify terms in the λ-calculus. So with β-reduction, we can apply an abstraction to an argument. To do this, we need to define substitution, free and bound variables.

2.3.1 Free and Bound Variables

Let us first define free and bound variables. Let P be any term in the λ-calculus, $FV(P)$ be the set that contains all free variables in P and FB be the set that contains all bound variables in P. We define both FV and BV inductively on P by Definition 2.5 and Definition 2.6. Since P can only be a variable, an abstraction or a function application, P can only be of the form x, $\lambda x.M$ and MN. Therefore, we define FV and BV for x, $\lambda x.M$ and MN.

Definition 2.5. FB

$$\begin{aligned} FV(x) &= \{x\} \\ FV(\lambda x.M) &= FV(M)\backslash\{x\} \\ FV(MN) &= FV(M) \cup FV(N) \end{aligned}$$

Let us now explain FV. The variable x in the λ-term x is free. All free variables in an abstraction are those that are not the parameter of the abstraction. For an application MN, all free variables are all free variables of λ-term M and λ-term N.

Definition 2.6. BV

$$\begin{aligned} BV(x) &= \emptyset \\ BV(\lambda x.M) &= BV(M) \cup \{x\} \\ BV(MN) &= BV(M) \cup BV(N) \end{aligned}$$

Let us now explain BV. The variable x in the λ-term x is a free variable, and the set of bound variables is empty. An abstraction $\lambda x.M$ binds the variable x. So the set of bound variables in an abstraction is the set of bound variables in M plus the variable x. For an application MN, the bound variables are all bound variables of both λ-term M and λ-term N.

We give two examples to illustrate the definitions of free and bound variables. We show that λ-term $\lambda x.yx$ has one free variable, y and one bound variable x.

Example 2.7. FV

$$\begin{aligned} FV(\lambda x.yx) &= FV(yx)\backslash\{x\} \\ &= (FV(x) \cup FV(y))\backslash\{x\} \\ &= (\{x\} \cup \{y\})\backslash\{x\} \\ &= \{x, y\}\backslash\{x\} \\ &= \{y\} \end{aligned}$$

Example 2.8. BV

$$\begin{aligned} BV(\lambda x.xy) &= BV(xy) \cup \{x\} \\ &= (BV(x) \cup BV(y)) \cup \{x\} \\ &= (\emptyset \cup \emptyset) \cup \{x\} \\ &= \{x\} \end{aligned}$$

The following example will demonstrate that using two different variables with the same name in a λ-term is quite tricky: $(\lambda x.x)x$. By the definitions of FV and BV, we can reason that $FV((\lambda x.x)x) = \{x\}$ and $BV((\lambda x.x)x) = \{x\}$. While these sets are the same, they do not refer to the same x. This is quite confusing, and we prefer to write $(\lambda y.y)x$ instead, where y is a new "fresh" variable with a different name. We now have $FV((\lambda y.y)x) = \{x\}$ and $BV((\lambda y.y)x) = \{y\}$. These λ-terms are equivalent by the notion of α-equivalence, which states that λ-terms that differ only by the names of bound variables are considered the same. We discuss this in more detail after we define substitution.

2.3.2 Substitution

Keeping the definitions of free and bound variables in mind, we now define substitution. In this thesis, substitution is denoted as $P[x \mapsto Q]$ and is defined inductively on P by Definition 2.9.

Definition 2.9. Substitution

$x[x \mapsto Q] = Q$
$y[x \mapsto Q] = y$ if $(x \neq y)$
$(MN)[x \mapsto Q] = M[x \mapsto Q]N[x \mapsto Q]$
$(\lambda x.M)[x \mapsto Q] = \lambda x.M$
$(\lambda y.M)[x \mapsto Q] = \lambda z.M[y \mapsto z][x \mapsto Q]$ if $(x \neq y)$

with z a variable defined by:

1. If $x \notin FV(N)$ or $y \notin FV(M)$ then $z = y$
2. Otherwise, z can be any variable such that $z \notin FV(N)$ or $z \notin FV(M)$

The variable z always exists. Choosing a fresh variable for z always satisfies condition two. It might not be clear why we need to use the variable z and one might expect the rule in Definition 2.10 is sufficient.

Definition 2.10. Incorrect Substitution Rule

$$(\lambda y.M)[x \mapsto Q] = \lambda z.M[x \mapsto Q] \text{ if } (x \neq y)$$

However, the following example illustrates why the more complicated rule is necessary. Consider the substitution $(\lambda y.xy)[x \mapsto y]$. The free variable of λ-term $\lambda y.xy$ is x and the bound variable is y. If we use the simple rule above, we get that $(\lambda y.xy)[x \mapsto y] = \lambda y.yy$, the meaning of the λ-term changes. However, the free variable we substituted, is not free anymore. The variable y is not a free variable in the term that we substituted y in. Therefore, we change the bound variable y in $\lambda y.xy$ to a new variable z. We can pick z as z is not a free variable in xy. Following the more complex rule, we get:

$$\begin{aligned}(\lambda y.xy)[x \mapsto y] &= \lambda z.(xy)[y \mapsto z][x \mapsto y] \\ &= \lambda z.yz\end{aligned}$$

2.3.3 α-equivalence

Here, we define α-equivalence. The idea is that renaming bound variables does not change the meaning of a λ-term. For example, the λ-terms $\lambda x.x$ and $\lambda y.y$ are equivalent ($\lambda x.x \equiv_\alpha \lambda y.y$), since renaming all bound variables can result in identical λ-terms. The λ-terms $\lambda x.x$ and $\lambda y.y$ represent the same idea. We define the relation $P \equiv_\alpha P'$ inductively on P and P' in Definition 2.11.

Definition 2.11. α-equivalence

$x \equiv_\alpha y$	if $x = y$
$MN \equiv_\alpha M'N'$	if $M \equiv_\alpha M'$ and $N \equiv_\alpha N'$
$\lambda x.M \equiv_\alpha \lambda y.M'$	if $M \equiv_\alpha M'[y \mapsto x]$ with $x = y$ or $x \notin FV(M')$

The last rule states that abstractions $\lambda x.M$ and $\lambda y.M'$ are equivalent if M and M' are equivalent after substituting y for x in M'. However, x should be equal to y or x should not be a free variable in M'.

Example 2.12. Consider the following:

- $\lambda x.zx$ is equivalent to $\lambda y.zy$, since $zy[y \mapsto x] = zx \equiv_\alpha zx$ and x is not a free variable in zy.
- $\lambda z.zx$ is not equivalent to $\lambda y.zy$, since z is a free variable in zy and x is not equal to y.

2.3.4 β-Reduction

Now, we are ready to discuss β-reduction. In the λ-calculus, there is only one way to simplify terms, which is β-reduction. This β-reduction relation is defined in Definition 2.13 with M and N being terms in the λ-calculus.

Definition 2.13. β-reduction

$$(\lambda x.M)N \to M[x \mapsto N] \qquad (\beta)$$

A λ-term of the form $(\lambda x.M)N$ can be reduced by substituting x by N in M. However, without additional rules, we cannot apply β-reduction to subterms. For example, the following reduction would not be possible: $\lambda x.(\lambda y.y)x \to \lambda x.x$. Therefore, in Definition 2.14, we define the following closure rules: μ, ν and ξ.

Definition 2.14. Closure rules for β-reduction

$$\frac{M \to M'}{MN \to M'N}\,(\mu) \qquad \frac{N \to N'}{MN \to MN'}\,(\nu) \qquad \frac{M \to M'}{\lambda x.M \to \lambda x.M'}\,(\xi)$$

Now, we define $\to_\beta$ as β closed under μ, ν and ξ. So with $\to_\beta$ we can use β-reduction on all subterms if the subterm is of the form $(\lambda x.M)N$. A subterm of the form $(\lambda x.M)N$ is called a β-redex. In one reduction, we may need to use multiple rules. The β-redex that is reduced is underlined. For instance, we need to use the μ and ξ rule in the last example of the following reductions:

Example 2.15. Usage of Closure Rules

$$\begin{aligned}
(\underline{\mathbf{II}})(\mathbf{II}) &\to_\beta \mathbf{I}(\mathbf{II}) && \beta \text{ with } \mu \text{ rule}\\
(\mathbf{II})(\underline{\mathbf{II}}) &\to_\beta (\mathbf{II})\mathbf{I} && \beta \text{ with } \nu \text{ rule}\\
\lambda x.\underline{\mathbf{I}x} &\to_\beta \lambda x.x && \beta \text{ with } \xi \text{ rule}\\
(\mathbf{I}(\lambda x.\underline{\mathbf{I}x}))(\mathbf{II}) &\to_\beta (\mathbf{I}(\lambda x.x))(\mathbf{II}) && \beta \text{ with } \mu \text{ and } \xi \text{ rule}
\end{aligned}$$

We see that β-reduction is a non-deterministic relation, which means that there can be multiple ways to reduce a term. However, sometimes we would only like to reduce a term in one way and have a deterministic relation. To achieve this, we can define subrelations that do not allow all closure rules.

2.4 Call-by-name λ-calculus

In this section, we discuss the call-by-name λ-calculus (λ_n). The reduction rule that is used in this calculus is the same as the β reduction rule. However,

we name the rule β_n (Definition 2.16) to make it clear that it is the reduction rule of λ_n. While call-by-name avoids evaluating the argument of a function, arguments are re-evaluated every time they are used in the body of the function. Lazy evaluation stores the evaluation of each argument, so it does not have to re-evaluate the same argument every time it is used. Therefore, an optimised version of call-by-name evaluation is lazy evaluation, which is used in many programming languages. However, for simplicity, we will only focus on call-by-name.

Definition 2.16. β_n-reduction

$$(\lambda x.M)N \to M[x \mapsto N] \qquad (\beta_n)$$

We define the relation $\to_{\beta_n}$ as β_n closed under μ, ν and ξ. The relation $\to_{\beta_n}$ is the call-by-name reduction relation, which allows reduction of λ-terms under all term formers. Weak reduction, $\to_w$, is defined as β_n closed under μ and ν. The only restriction of $\to_w$ is that it cannot reduce under λ's. Weak reduction will be used later to define a deterministic relation that supports cbn and cbv evaluation in a different λ-calculus. We also define the relation $\to_n$ as β_n closed under μ. So the relation $\to_n$ cannot evaluate the argument of a function, since the ν rule is not allowed with $\to_n$. Call-by-name evaluation is thus defined as $\to_n^*$.

Table 2.1 shows the closure rules for the different reduction relations we have defined thus far. In Example 2.17, we give the complete cbn evaluation of λ-term $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{II})$. The β-redex that is reduced is underlined in each step. Note that $\to_n$ is deterministic, so there is always only one way to apply $\to_n$ on a λ-term.

Relation	$\boldsymbol{\beta}$	$\boldsymbol{\mu}$	$\boldsymbol{\nu}$	$\boldsymbol{\xi}$
$\to_{\beta_n}$	β_n	✓	✓	✓
$\to_n$	β_n	✓		
$\to_w$	β_n	✓	✓	

Table 2.1: Closure Rules for Different Reduction Relations

Example 2.17. Cbn evaluation of $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{II})$

$$\begin{aligned}(\underline{\mathbf{I}(\lambda x.\mathbf{I}x)})(\mathbf{II}) &\to_n \underline{(\lambda x.\mathbf{I}x)(\mathbf{II})} \\ &\to_n \underline{\mathbf{I}(\mathbf{II})} \\ &\to_n \underline{\mathbf{II}} \\ &\to_n \mathbf{I}\end{aligned}$$

2.5 Call-by-value λ-calculus

Before we define the call-by-value λ-calculus (λ_v), we first define the notion of values. Values are λ-terms that are fully evaluated. That is, values are λ-terms that cannot be reduced any further and thus do not contain any function applications. We define values Definition 2.18 as all λ-terms that are either a variable or an abstraction. We denote values by V or W.

Definition 2.18. Values in λ_v

$$V \quad ::= \quad x \mid \ \lambda x.M$$

The main idea of λ_v is that we only reduce a function application if the argument is a value. The β-reduction rule is defined in Definition 2.19.

Definition 2.19. β_v-reduction

$$(\lambda x.M)V \to M[x \mapsto V] \qquad (\beta_v)$$

We define a new closure rule, $\nu_<$, which is similar to the ν rule, but requires the left λ-term to be a value. In call-by-value evaluation, we first reduce the left λ-term of a function application, to an abstraction, before reducing the right λ-term to a value to apply β_v.

Definition 2.20. $\nu_<$

$$\frac{N \to N'}{VN \to VN'}\,(\nu_<)$$

We define call-by-value reduction, $\to_{\beta_v}$, as β_v closed under μ, ν and ξ which allows reduction of λ-terms under all term formers. The relation $\to_v$ is defined as β_v, closed under μ and $\nu_<$. Table 2.2 shows the closure rules for all different reduction relations we have defined thus far.

Relation	β	μ	ν	$\nu_<$	ξ
$\to_{\beta_n}$	β_n	✓	✓	✓	✓
$\to_n$	β_n	✓			
$\to_w$	β_n	✓	✓		
$\to_{\beta_v}$	β_v	✓	✓	✓	✓
$\to_v$	β_v	✓		✓	

Table 2.2: Closure Rules for Different Reduction Relations

Note that $\to_v$ is deterministic, so there is only one way to apply $\to_v$ on a λ-term. The choice between using μ, $\nu_<$ or no closure rule for $\to_v$ is deterministic. Consider the λ-term MN.

1. If M is an abstraction and N is a value, we can only apply β_v directly.
2. If M is an abstraction and N is an application, we can only apply $\nu_<$ to reduce the left λ-term.
3. If M is an application and N any λ-term, we can only apply μ to reduce the left λ-term.

The closure rule $\nu_<$ forces evaluation from left to right, hence $<$ in the name. First the closure rule μ is used after which $\nu_<$ is used. In Example 2.21, we give the cbv evaluation of λ-term $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{II})$. Again, the β-redex that is reduced is underlined.

Example 2.21. Cbv evaluation of $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{II})$

$$\begin{array}{rcl} (\underline{\mathbf{I}(\lambda x.\mathbf{I}x)})(\mathbf{II}) & \to_v & (\lambda x.\mathbf{I}x)(\underline{\mathbf{II}}) \\ & \to_v & \underline{(\lambda x.\mathbf{I}x)\mathbf{I}} \\ & \to_v & \underline{\mathbf{II}} \\ & \to_v & \mathbf{I} \end{array}$$

Chapter 3

Call-by-box Lambda Calculus

3.1 Specification of λ_b

3.1.1 Intuitionistic logic IS4

Intuitionistic logic IS4 is a non-classical logic that is based on the idea of constructive proof. Unlike classical logic, intuitionistic logic does not allow for the law of excluded middle, which states that for every proposition P, the statement $P \vee \neg P$ assumed. In intuitionistic logic, a proposition is only considered true if it can be constructively proven. That is, a proposition is true if we can explicitly construct a proof, much like writing a program. This is why programming languages and proofs in systems like Agda and Coq are based on intuitionistic logic. Since λ-calculus is the foundation of functional programming, it corresponds with intuitionistic logic via the Curry-Howard correspondence, where programs are equivalent to proofs and types to propositions.

The intuitionistic modal logic IS4 is an extension of intuitionistic logic that introduces *modal operator* $\Box$. The $\lambda_\Box$ calculus is a fragment of IS4 that, besides the usual λ-terms, also uses box (box) and unbox (ε) operators and is strong enough to interpret both call-by-name (cbn) and call-by-value (cbv) lambda calculi [1]. The box is like a little container that delays a computation, making it possible to simulate both cbn and cbv evaluation strategies. In this thesis, we formalize the *boxed lambda calculus* in Agda, denoted as λ_b, which is as a refinement of the simpler $\lambda_\Box$ calculus.

3.1.2 Terms and Types of λ_b

We now define the types of the λ_b calculus. We indicate the types of λ_b with A and B, representing types and boxed types respectively. The type

X denotes the base type. Note that functions types are of the form $B \to A$ thus the input is always a boxed type.

Definition 3.1. Types of λ_b

$$A \quad ::= \quad X \mid \ B \to A \mid \ B \qquad\qquad B \quad ::= \quad \Box A$$

We give some examples of types in λ_b. Note that A can be any type that is not boxed, while B can be any type that is boxed. The type X is the base type.

Example 3.2. Types in λ_b

$$X$$
$$\Box(B \to A)$$
$$\Box X \to X$$
$$\Box A \to \Box B$$
$$\Box\Box\Box X \to \Box X$$

Since variables are always boxed, variables in λ_b are always inside the unboxing operator ε. Thus, we only have $\varepsilon(x)$ in the grammar with x a variable. Abstractions in λ_b, take boxed terms as arguments, which is why we define abstractions as $\lambda x.M$, where x is the parameter of a boxed type and M is a term. The rest of the terms are defined in Definition 3.3 and some example terms are given in Example 3.4.

Definition 3.3. Terms in λ_b

$$M, N, P, Q \quad ::= \quad \varepsilon(x) \mid \ \lambda x.M \mid \ MN \mid \ \mathsf{box}(N)$$

Example 3.4. Terms in λ_b

$$\varepsilon(x)$$
$$\lambda x.\varepsilon(x)$$
$$(\lambda x.\varepsilon(x))\mathsf{box}(\varepsilon(y))$$
$$(\lambda x.\varepsilon(x))\mathsf{box}(\lambda y.\varepsilon(y))$$

The typing rules of λ_b are defined in Definition 2.3. The $\Box$-I rule states that if M is of type A, then $\mathsf{box}(M)$ is of type $\Box A$. The $\mathrm{Id}_\Box$ rule is the identity rule for boxed variables, which states that if we know that x is a variable of type $\Box A$, we can conclude that $\varepsilon(x)$ is of type A. The only way to add variables to the context is by using the $\to$-I rule. These variables are always boxed, since they are the parameters of abstractions. Due to the grammar of λ_b, all variables in λ_b occur inside the unboxing operator ε and thus the $\mathrm{Id}_\Box$ rule covers all cases. The $\to$-E rule states that if we have a term M of type $B \to A$ and a term N of type B, we can conclude that the application MN has type A. It is important that the argument N is a boxed term, which is why we denote its type as B.

Definition 3.5. Typing rules of λ_b

$$\frac{}{\Gamma, x : \Box A \vdash \varepsilon(x) : A}\ \mathrm{Id}_\Box$$

$$\frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x.M : A \to B}\ \to\text{-I}$$

$$\frac{\Gamma \vdash M : B \to A \qquad \Gamma \vdash N : B}{\Gamma \vdash MN : B}\ \to\text{-E}$$

$$\frac{\Gamma \vdash M : A}{\Gamma \vdash \mathsf{box}(M) : \Box A}\ \Box\text{-I}$$

We now demonstrate these typing rules with Example 3.6, where we show that the term $(\lambda x.\varepsilon(x))\mathsf{box}(\lambda y.\varepsilon(y))$ is well-typed. Note that the argument $\mathsf{box}(\lambda y.\varepsilon(y))$ is boxed. If the argument was not boxed, we would not be able to apply the abstraction $\lambda x.\varepsilon(x)$ to it.

Example 3.6. Well-typedness of $(\lambda x.\varepsilon(x))\mathsf{box}(\lambda y.\varepsilon(y))$

$$\dfrac{\dfrac{\dfrac{}{x : \Box(\Box A \to A) \vdash \varepsilon(x) : \Box A \to A}\ \mathrm{Id}_\Box}{\vdash \lambda x.\varepsilon(x) : \Box(\Box A \to A) \to (\Box A \to A)}\ \to\text{-I} \qquad \dfrac{\dfrac{\dfrac{}{y : \Box A \vdash \varepsilon(y) : A}\ \mathrm{Id}_\Box}{\vdash \lambda y.\varepsilon(y) : \Box A \to A}\ \to\text{-I}}{\vdash \mathsf{box}(\lambda y.\varepsilon(y)) : \Box(\Box A \to A)}\ \Box\text{-I}}{\vdash (\lambda x.\varepsilon(x))\mathsf{box}(\lambda y.\varepsilon(y)) : \Box A \to A}\ \to\text{-E}$$

3.2 Substitution and Free and Bound variables in λ_b

Since variables always occur inside ε, we need to redefine free and bound variables and substitution. For λ_b, we have:

Definition 3.7 (Free and Bound Variables)**.** The free and bound variables of a term M are defined by:

$$\begin{array}{llll} FV_b(\varepsilon(x)) & = \{\varepsilon(x)\} & BV(\varepsilon(x)) & = \emptyset \\ FV_b(\lambda x.M) & = FV(M)\backslash\{\varepsilon(x)\} & BV(\lambda x.M) & = BV(M) \cup \{\varepsilon(x)\} \\ FV_b(MN) & = FV(M) \cup FV(N) & BV(MN) & = BV(M) \cup BV(N) \end{array}$$

Definition 3.8 (Substitution in λ_b)**.** The substitution of a term Q for a variable $\varepsilon(x)$ in a term M is defined as follows:

$$\begin{aligned}
\varepsilon(x)[\varepsilon(x) \mapsto Q] &= Q \\
\varepsilon(y)[\varepsilon(x) \mapsto Q] &= \varepsilon(y) \text{ if } (x \neq y) \\
\mathsf{box}(M)[\varepsilon(x) \mapsto Q] &= \mathsf{box}(M[\varepsilon(x) \mapsto Q]) \\
(MN)[\varepsilon(x) \mapsto Q] &= M[x \mapsto Q]N[x \mapsto Q] \\
(\lambda x.M)[\varepsilon(x) \mapsto Q] &= \lambda x.M \\
(\lambda y.M)[\varepsilon(x) \mapsto Q] &= \lambda z.M[\varepsilon(y) \mapsto \varepsilon(z)][\varepsilon(x) \mapsto Q] \text{ if } (x \neq y)
\end{aligned}$$

with z a variable defined by:

1. If $\varepsilon(x) \notin FV_b(N)$ or $\varepsilon(y) \notin FV_b(M)$ then $z = y$
2. Otherwise, z can be any variable such that $\varepsilon(z) \notin FV(N)$ or $\varepsilon(z) \notin FV(M)$

With this new lambda calculus, it may not be completely clear that substitution preserves well-typedness. We can prove that substitution preserves well-typedness in λ_b.

Proposition 3.9 (Preservation of well-typedness)**.**

$$\Gamma, x : \Box A \vdash M : B \text{ and } \Gamma \vdash Q : A \implies \Gamma \vdash M[\varepsilon(x) \mapsto Q] : B$$

Proof. We prove by structural induction on the term M that this substitution preserves well-typedness.

1. $$\varepsilon(x)[\varepsilon(x) \mapsto Q] = Q$$

   Since Q is well-typed, and Q is the result of the substitution, we have that well-typedness is preserved.

2. $$\varepsilon(y)[\varepsilon(x) \mapsto Q] = \varepsilon(y) \quad \text{if } x \neq y$$

   We have that $x \neq y$, and the result of the substitution is $\varepsilon(y)$, which is well-typed.

3. $$\mathsf{box}(M)[\varepsilon(x) \mapsto Q] = \mathsf{box}(M[\varepsilon(x) \mapsto Q])$$

   We have that $\mathsf{box}(M)$ is well-typed and thus M is well-typed because of the $\Box$-I rule. By the inductive hypothesis, we have that $M[\varepsilon(x) \mapsto Q]$ is well-typed. We also have that $\mathsf{box}(M[\varepsilon(x) \mapsto Q])$ is well-typed because of the $\Box$-I rule. Thus, we have that $\mathsf{box}(M)[\varepsilon(x) \mapsto Q]$ is well-typed.

4.
$$(MN)[\varepsilon(x) \mapsto Q] = (M[\varepsilon(x) \mapsto Q])\ (N[\varepsilon(x) \mapsto Q])$$

We have that MN is well-typed and thus M and N are well-typed. By the inductive hypothesis, we get that $M[\varepsilon(x) \mapsto Q]$ is well-typed, and $N[\varepsilon(x) \mapsto Q]$ is well-typed. We also have that application of two well-typed terms is well-typed, so we have that $(MN)[\varepsilon(x) \mapsto Q]$ is well-typed.

5.
$$(\lambda x.M)[\varepsilon(x) \mapsto Q] = \lambda x.M$$

We have that $\lambda x.M$ is well-typed and thus M is well-typed. Since $\varepsilon(x)$ is a bound variable in $\lambda x.M$, no substitution occurs and the result is the well-typed term $\lambda x.M$.

6.
$$(\lambda y.M)[\varepsilon(x) \mapsto Q] = \lambda z.(M[\varepsilon(y) \mapsto \varepsilon(z)][\varepsilon(x) \mapsto Q]) \quad \text{if } x \neq y$$

We have that $\lambda y.M$ is well-typed and thus M is well-typed. By the inductive hypothesis, we have that $M[\varepsilon(y) \mapsto \varepsilon(z)]$ and $M[\varepsilon(y) \mapsto \varepsilon(z)][\varepsilon(x) \mapsto Q]$ are well-typed. We also have that abstraction of a well-typed term is well-typed, so we have that
$\lambda z.(M[\varepsilon(y) \mapsto \varepsilon(z)][\varepsilon(x) \mapsto Q])$ is well-typed. □

With this definition of substitution, we define the reduction rule we can use for this λ_b.

Definition 3.10 (β_b)**.** In λ_b, β reduction is defined as follows:

$$(\lambda x.M)\mathsf{box}(N) \to M[\varepsilon(x) \mapsto N] \qquad (\beta_b)$$

In λ_b, the arguments of abstractions are always boxed, so we can only apply abstractions to boxed terms, which makes sense since the parameters like x are always boxed. Since variables only occur insides ε, we can replace those with the corresponding unboxed term.

We define three relations: β_b closed under μ and ν, β_b closed under μ and β_b closed under ν. We call them $\to_{we}$, $\to_{b_n}$ and $\to_{b_v}$ respectively. The relation $\to_{we}$ is weak since values do not reduce. That is, abstractions and terms of the form $\varepsilon(x)$ do not reduce. The relation $\to_{we}$ is also external as boxes do not reduce. This relation is called call-by-box (cbb) evaluation. We will see after defining the embeddings that $\to_{we}$ collapses to $\to_{b_n}$ in Girard's embedding and that $\to_{we}$ collapses to $\to_{b_v}$ in Gödel's embedding.

Relation	$\boldsymbol{\beta}$	$\boldsymbol{\mu}$	$\boldsymbol{\nu}$	$\boldsymbol{\nu_<}$	$\boldsymbol{\xi}$
$\rightarrow_{\beta_n}$	β_n	✓	✓	✓	✓
$\rightarrow_n$	β_n	✓			
$\rightarrow_w$	β_n	✓	✓		
$\rightarrow_{\beta_v}$	β_v	✓	✓	✓	✓
$\rightarrow_v$	β_v	✓		✓	
$\rightarrow_{\beta_b}$	β_b	✓	✓	✓	✓
$\rightarrow_{we}$	β_b	✓	✓	✓	
$\rightarrow_{b_n}$	β_b	✓			
$\rightarrow_{b_v}$	β_b		✓		

Table 3.1: Closure Rules for Different Reduction Relations

Chapter 4

Embeddings in λ_b

4.1 Girard's Embedding

Here we present Girard's embedding, which is a method to represent a logical system in a different framework, often used in the context of linear logic and proof theory.

4.2 Embedding of λ_n into λ_b

Let Λ_{CBN} be the set containing all λ-terms in the λ_n and let Λ_{CBB} be the set containing all λ-terms in the λ_b. The set $\mathcal{T}_{CBN}$ contains all types of the λ_n and $\mathcal{T}_{CBB}$ contains all types of the λ_b. Now we define functions from Λ_{CBN} to Λ_{CBB} for term translation and from $\mathcal{T}_{CBN}$ to $\mathcal{T}_{CBB}$ for type translation. These functions represent an embedding of the λ_n into the λ_b.

$$\begin{aligned} X^\circ &= X & x^\circ &= \varepsilon(x) \\ (A_1 \to A_2)^\circ &= \Box {A_1}^\circ \to {A_2}^\circ & (\lambda x.M)^\circ &= \lambda x.M^\circ \\ & & (MN)^\circ &= M^\circ \mathsf{box}(N^\circ) \end{aligned}$$

Figure 4.1: Girard's translation from λ_n to λ_b.

The image of Girard's translation is a subset of the λ_b given by the following grammar:

$$M, N \quad ::= \quad \varepsilon(x) \mid \ \lambda x.M \mid \ M\mathsf{box}(N)$$

Since application in Girard's translation is of the form

$$M\mathsf{box}(N)$$

reducing the left-hand side of the application is not possible without the ζ rule that allows reduction inside boxed terms.

To clarify, we give the embedding of $\mathbf{I}x$ into λ_b after which we apply $\rightarrow_{b_n}$. It is clear that the result obtained after $\rightarrow_{b_n}$ is equal to the embedding of the result of applying $\rightarrow_{b_n}$ on $\mathbf{I}x$ in the λ_n, which is $T(x) = \varepsilon(x)$.

$$\begin{array}{rcl}
(\mathbf{I}x)^\circ & = & (\lambda x.x)^\circ \mathsf{box}(x^\circ) \\
& = & (\lambda x.x^\circ)\mathsf{box}(x^\circ) \\
& = & (\lambda x.\varepsilon(x))\mathsf{box}(\varepsilon(x)) \\
& \rightarrow_{b_n} & (\lambda x.\varepsilon(x))[\varepsilon(x) \mapsto \varepsilon(x)] \\
& = & \varepsilon(x)
\end{array}$$

4.3 Gödel's Embedding

Here we present Gödel's embedding, which is a method to represent a logical system in a different framework, often used in the context of intuitionistic logic and proof theory.

For the embedding of the λ_v into the λ_b, we make use of the abbreviation "raise" defined by:

$$\mathsf{raise}(M) := \lambda z.\varepsilon(z)M$$

Let Λ_{CBV} be the set containing all λ-terms in the λ_v and let Λ_{CBB} be the set containing all λ-terms in the λ_b. The set $\mathcal{T}_{CBV}$ contains all types of the λ_v and $\mathcal{T}_{CBB}$ contains all types of the λ_b. Now we define functions from Λ_{CBV} to Λ_{CBB} for term translation and from $\mathcal{T}_{CBV}$ to $\mathcal{T}_{CBB}$ for type translation.

$$\begin{array}{cc}
A^* = \Box A^\bullet & V^* = \mathsf{box}(V^\bullet) \\
 & (MN)^* = \mathsf{raise}(N^*)M^*
\end{array}$$

$$\begin{array}{cc}
X^\bullet = X & x^\bullet = \varepsilon(x) \\
(A_1 \rightarrow A_2)^\bullet = \Box A_1{}^\bullet \rightarrow \Box A_2{}^\bullet & (\lambda x.M)^\bullet = \lambda x.M^*
\end{array}$$

Figure 4.2: Gödel's translation from λ_v to λ_b.

To clarify, we give the embedding of $\mathbf{I}x$ into λ_b after which we apply $\rightarrow_{b_v}$ twice. It is clear that the result obtained after $\rightarrow_{b_v}$ is equal to the embedding of the result of applying $\rightarrow_v$ on $\mathbf{I}x$ in the λ_v, which is $T(x) = \mathsf{box}(\varepsilon(x))$.

$$
\begin{array}{rcl}
T(\mathbf{I}x) & = & \mathsf{raise}(T(x))T(\mathbf{I}) \\
& = & \mathsf{raise}(T(x))T(\lambda x.x) \\
& = & \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.T(x)) \\
& = & \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.\mathsf{box}(\varepsilon(x))) \\
& = & (\lambda z.\varepsilon(z)\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.\mathsf{box}(\varepsilon(x))) \\
& \to_{b_v} & (\lambda x.\mathsf{box}(\varepsilon(x)))\mathsf{box}(\varepsilon(x)) \\
& \to_{b_v} & \mathsf{box}(\varepsilon(x))
\end{array}
$$

Chapter 5

Formalisation in Agda

5.1 Box Calculus

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

5.2 Linear Logic

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec tincidunt, nunc ut bibendum facilisis, nisi nunc fringilla nisi, nec mollis nunc nisi nec ligula. Donec id ligula ac enim efficitur mollis. Donec id ligula ac enim efficitur mollis. Donec id ligula ac enim efficitur mollis. Donec id ligula ac enim efficitur mollis.

Chapter 6

Related Work

In this chapter you demonstrate that you are sufficiently aware of the state-of-art knowledge of the problem domain that you have investigated as well as demonstrating that you have found a *new* solution / approach / method.

Chapter 7

Conclusions

In this chapter you present all conclusions that can be drawn from the preceding chapters. It should not introduce new experiments, theories, investigations, etc.: these should have been written down earlier in the thesis. Therefore, conclusions can be brief and to the point.

Bibliography

[1] José Espírito Santo, Luís Pinto, and Tarmo Uustalu. Modal Embeddings and Calling Paradigms. In Herman Geuvers, editor, *4th International Conference on Formal Structures for Computation and Deduction (FSCD 2019)*, volume 131 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 18:1–18:20, Dagstuhl, Germany, 2019. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik.

Appendix A

Appendix

Appendices are *optional* chapters in which you cover additional material that is required to support your hypothesis, experiments, measurements, conclusions, etc. that would otherwise clutter the presentation of your research.